

1、 进程 P0 和 P1 的共享变量定义及其初值为：

boolean falg[2];

int turn=0;

falg[0]=FALSE; falg[1]=FALSE;

若进程 P0 和 P1 访问临界资源的类 C 伪代码实现如下：

<pre>void P0() //进程 P0 {while(TRUE){ flag[0]= TRUE; turn=1; while(flag[1]&&turn==1); 临界区; flag[0]=FALSE; } }</pre>	<pre>void P1() //进程 P1 {while(TRUE){ flag[1]= TRUE; turn=0; while(flag[0]&&turn==0); 临界区; flag[1]=FALSE; } }</pre>
--	--

则并发执行进程 P0 和 P1 时产生的情形是

【全国联考 2010】

- A. 不能保证进程互斥进入临界区、会出现“饥饿”现象
- B. 不能保证进程互斥进入临界区、不会出现“饥饿”现象
- C. 能保证进程互斥进入临界区、会出现“饥饿”现象
- D. 能保证进程互斥进入临界区、不会出现“饥饿”现象

分析进程的执行过程：一开始，没有进程处于临界区中，现在进程 P0 开始执行，通过设置其数组元素和将 turn 置 1 来标识它希望进入临界区，由于进程 P1 并不想进入临界区，所以 P0 跳出 while 循环，进入临界区。如果进程 P1 现在开始执行，进程 P1 将阻塞在 while 循环直到 flag[0]变为 false，而该事件只有进程 P0 退出临界区时才会发生。

现在考虑两个进程几乎同时执行到 while 循环的情况，它们分别在 turn 中存入 1 和 0，但只有后被保存进去的进程号才有效，前一个被重写而丢失。假设进程 P1 是后存入的，则 turn 为 0。进程 P0 将循环 0 次而进入临界区，而进程 P1 则将不停地循环且不能进入临界区，直到进程退出临界区为止。

因此，该算法实现了临界区互斥。

“饥饿”出现的时机：使用忙等待实现互斥，当一个进程离开临界区时，如果有多个进程等待进入临界区，系统会随机选择一个进程执行，因为这种随机性，会导致有些进程长期得不到执行，因而导致“饥饿”。

本题中，如果 P1 已经等在 while 上的时候，P0 至多执行一次临界区，否则下次执行的时候，即便它在 P1 测试条件前出了临界区并重新设定了 flag，但由于它必须要设定 turn=1（此时 P1 不会再设置 turn 了），因此这样 P0 必然卡在 while 上，从而换到 P1 执行。所以不会出现“饥饿”现象。

2、 在一间酒吧里有三个音乐爱好者队列，第一个音乐爱好者只有随身听，第二个只有音乐磁带，第三个只有电池，而要听音乐就必须有随身听，音乐磁

带和电池这三中物品。酒吧老板一次出售这三种物品中的任意两种，当一名音乐爱好者得到这三种物品并听完乐曲后，酒吧老板才能再一次出售这三种物品中任意两种，于是第二名音乐爱好者得到这三种物品。并开始听乐曲，全部买卖就这样进行下去。使用 **P, V** 操作正确解决这一买卖。【北京大学 1999】

分析：同步信号量 **S** 表示是否听完乐曲，初值为 0；

资源信号量 **S1、S2、S3** 表示两种物品组合起来的临界资源，初值均为 0。

Var S, S1,S2,S3:semaphore;

Var flag1,flag2,flag3:Boolean; //依次代表随身听、磁带和电池

S:=0;

S1:=S2:=S3:=0;

Cobegin

process provider

begin

repeat

flag1:=flag2:=flag3:=False;

取两种物品，设置 **flag_i** 为 **True**;

if flag2&flag3 then V(S1);

elseif flag1&flag3 then V(S2);

else V(S3);

P(S);

untile false;

end

process1 //只有随身听

begin

repeat

P(S1);

购买物品；听乐曲；

V(S);

untile false;

end

process2 //只有磁带

begin

repeat

p(S2);

购买物品；听乐曲；

V(S);

untile false;

end

```

process3 //只有电池
begin
  repeat
    P(S3);
    购买物品; 听乐曲;
    V(S);
  until false;
end
Coend

```

3、某银行有人民币储蓄业务，由 n 个柜员负责，有 1 台取号机。每个顾客进入银行后先取一个号，若有人取号则需等他人取完后才能取，取到号后等待叫号，当一个柜员人员空闲下来，就叫下一个号。试用 P ， V 操作正确编写柜台人员和顾客进程的程序。【昆明理工大学 2006】

【分析】顾客相当于生产者，柜员相当于消费者，所有顾客领取号码后就进入了一个等待队列，该等待队列相当于 **buffer**。这个问题基本符合一般意义的“生产者—消费者”问题，但又有所不同，不同在于顾客（即生产者）“取号进入等待队列”操作不需要与柜员（消费者）同步（不需等待空的缓冲区）。

只需要两个信号量即可，一个用于互斥访问等待队列（对于顾客，就是互斥使用柜员机取号；对于柜员，就是叫号时候互斥访问等待队列），一个用于柜员“叫号”操作与顾客的同步（需测试是否有人在等）。

```

Var mutex=1, customer_count=0: semaphore;

```

```

Cobegin

```

```

  process customer

```

```

    begin

```

```

      repeat

```

```

        p(mutex);

```

```

        取号码, 进入队列;

```

```

        v(mutex);

```

```

        v(customer_count);

```

```

      until false;

```

```

    end

```

```

  process server(i=1,...,n)

```

```

    begin

```

```

      repeat

```

```

        p(customer_count);

```

```

        p(mutex);

```

```

        从队列中取下一个号码;

```

```

        v(mutex);

```

```

        为该号码持有者服务;

```

```

      until false;

```

```

    end

```

```

Coend

```

4、有一阅览室，读者进入时必须先在一张登记表上登记。该表中每个表项代表阅览室中的一个座位。读者离开时要消掉其登记信息。阅览室共有 **50 个座位**。登记表**每次仅允许一位读者进行登记或注销**。读者登记时，发现登记表满，他在阅览室外等待，直至有空位再登记进入。试用类 **Pascal** 语言和 **P、V** 操作，描述读者行为。【国防科技大学 2000】(注：【南昌大学 2002】类似)

答：信号量 **seats**：表示可用座位数，初值为 50；

信号量 **mutex**：表示登记表是否正在使用，初值为 1；

```
var seats,mutex:semaphore;
    seats:=50;
    mutex:=1;
cobegin
    procedureEnter
    begin
        while(TRUE)
        begin
            p(seats);
            p(mutex);
            填写登记表;
            v(mutex);
            进入阅览室阅读;
        end
    end

    procedureLeave
    begin
        while(TRUE)
        begin
            p(mutex);
            消掉登记;
            v(mutex);
            离开阅览室;
            v(seats);
        end
    end
coend
```

5、某车站售票厅，任何时刻最多可容纳20名购票者进入，当售票厅中少于20名购票者时，则厅外的购票者可立即进入，否则需在外面等待。若把一个购票者看作一个进程，请回答下列问题

(1)用PV操作管理这些并发进程时，应怎样定义信号量，写出信号量的初值以及信号量各种取值的含义。

(2)若欲购票者最多为n个人，写出信号量可能的变化范围(最大值和最小值)。

答：

定义信号量S,初值为20。当 $s > 0$ 时，它表示可以继续进入购票厅的人数；当 $s = 0$ 时，表示厅内已有20人正在购票；当 $s < 0$ 时， $|S|$ 表示正等待进入的人数。


```

var S:semaphore;
S=20;
cobegin
    procedure P_i:
    begin
        p(s);
        .
        Enter and buy ticket;
        .
        v(s);
    end
coend
(2)最大值为20, 最小值为20-N。

```

作业 1: 桌上有 1 空盘, 最多可以容纳两个水果, 每次只能放入或取出 1 个水果。爸爸专向盘中放苹果, 妈妈专向盘中放桔子。儿子专等吃盘中的桔子, 女儿专等吃盘中的苹果。请用 **Wait()**、**Signal()**原语实现爸爸、妈妈、儿子、女儿之间的同步与互斥关系。

分析: 本题需设置 4 个信号量, 其中 **empty** 表示还可以向盘中放几个水果, 其初值为 2; **apple** 对应已放入盘中的苹果, **orange** 对应已放入盘中的橘子, 它们的初值均为 0; **mutex** 用来实现对盘子的互斥访问, 其初值为 1。

```

Main(0
Cobegin{
Father()
{ while(1)
    { p(empty);
      P(mutex);
      向盘中放水果;
      V(mutex);
      V(apple);
    }
}
Mather()
{ while(1)
    { p(empty);
      P(mutex);
      向盘中放水果;
      V(mutex);

```

```

        V(orange);
    }
    Son()
    { while(1)
      { p(orange);
        P(mutex);
        从盘中取橘子;
        V(mutex);
        V(empty);
        吃橘子;
      }
    }
  }
  daughter()
  { while(1)
    { p(apple);
      P(mutex);
      从盘中取橘子;
      V(mutex);
      V(empty);
      吃苹果;
    }
  }
}coend

```

【例 14】如图 2.7 所示，有多个 PUT 操作同时向 BUFF1 放数据，有一个 MOVE 操作不断地将 BUFF1 的数据移到 BUFF2，有多个 GET 操作不断地从 BUFF2 中将数据取走。BUFF1 的容量为 m ，BUFF2 的容量是 n ，PUT、MOVE、GET 每次操作一个数据，在操作的过程中要保证数据不丢失。试用 P、V 原语协调 PUT、MOVE 的操作，并说明每个信号量的含义和初值。

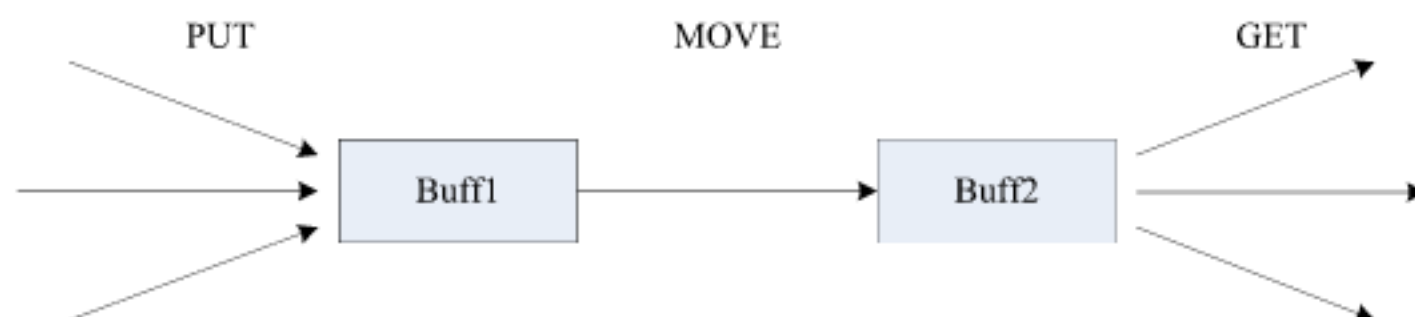


图 2.7 进程操作图

【分析】这里存在两个一般意义的“生产者—消费者”问题，PUT（生产者）与 MOVE（消费者）之间，需要设置三个信号量；MOVE（生产者）与 GET（消费者）之间，需要设置三个信号量。PUT 进程套用生产者进程即可，MOVE 进程只有在 Buff1 有新数据且 Buff2 有空闲区的时候才移动数据，GET 进程套用消费者进程即可。

答案：设置 6 个信号量 full1、empty1、B-M1、full2、empty2、B-M2，它们的含义和初值如下：

- 1) full1 表示 Buff1 是否有数据，初值为 0；
- 2) empty1 表示 Buff1 有空间，初值为 m；
- 3) B-M1 表示 Buff1 是否可操作，初值为 1；
- 4) Full2 表示 Buff2 是否有数据，初值为 0；
- 5) Empty2 表示 Buff2 有空间，初值为 n；
- 6) B-M2 表示 Buff2 是否可操作，初值为 1；

<PUT 类进程>

```
{
    repeat
        P(empty1);    /*判断 Buff1 是否有空间，没有则等待 */
        P(B-M1);      /*是否可操作 Buff1*/
        PUT;
        V(B-M1);      /*设置 Buff1 可操作标志 */
        V(full1);     /*设置 Buff1 有数据的标志 */
    until false
}
```

<MOVE 类进程>

```
{
    repeat
        P(full1);     /*判断 Buff1 是否有数据，没有则等待*/
        P(empty2);    /*判断 Buff2 是否有空间，没有则等待*/
        P(B-M1);      /*是否可操作 Buff1 */
        P(B-M2);      /*是否可操作 Buff2 */
        MOVE;
        V(B-M1);      /*设置 Buff1 可操作标志*/
        V(B-M2);      /*设置 Buff2 可操作标志*/
        V(empty1);    /*设置 Buff1 有空间标志*/
        V(full2);     /*设置 Buff2 有数据标志*/
    until false
}
```

<GET 类进程>

```
{
    repeat
        P(full2);     /*判断 Buff2 是否有数据，没有则等待 */
        P(B-M2);      /*是否可操作 Buff2*/
        GET;
        V(B-M2);      /*设置 Buff2 可操作标志 */
        V(empty2);    /*设置 Buff2 有空间的标志 */
    until false
}
```


作业:

1、下面是两个并发执行的进程，它们能正确执行吗?若不能，试举例说明，并修改之。【北航 2002】

```
Parbegin
  Var x:integer;
Process P1
  Var y,z:integer;
  Begin
    x:=1;
    y:=0;
    If x>=1 then y:=y+1;
    z:=y;
  End;
Process P2
  Var t,u:integer;
  Begin
    x:=0;
    t:=0;
    If x<=1 then t:=t+2;
    u:=t;
  End;
Parend;
```

2. 在公共汽车上，司机负责开车、停车和驾驶，售票员负责门的开门、关门和售票。基本操作规则是只有停车后，售票员才能开门，只有售票员关门后，司机才能开车。汽车初始状态处于行驶之中。当只有 1 个司机、2 个售票员、2 个门、每个售票员负责一个门时的协调操作。请使用 P、V 原语实现售票员与司机之间的协调操作，说明每个信号量的含义、初值和值的范围。【燕山大学 2006 复试】

3、某银行提供一个服务窗口和 10 个供顾客等待的座位。顾客到达银行时，若有空座位则到取号机上领取一个号，等待叫号。取号机每次仅允许一位顾客使用。当营业员空闲时，通过叫号选取一位顾客，并为其服务。顾客和营业员的活动过程描述如下：

```
Cobegin{
  Process 顾客 i
  {
    从取号机获得一个号码;
    等待叫号;
    获得服务;
```

```
}  
Process 营业员  
{  
    While(TRUE)  
    {  
        叫号;  
        为顾客服务;  
    }  
}  
}coend
```

请添加必要的信号量和 P、V（或 wait()、signal()）操作，实现上述过程中的互斥与同步。要求写出完整过程，说明信号量的含义并赋初值。（2011 联考）

